

Iterative Methods for Linear Systems

Lecture notes for MATH 616, Computational Linear Algebra
Dr. Xuping Xie, Old Dominion University

Fall 2026

Abstract

These notes develop the three classical iterative methods for a linear system $Ax = b$: the Jacobi method, the Gauss–Seidel method, and successive over-relaxation (SOR). Each method is presented component by component, as an algorithm in Python, and as a matrix iteration $x^{(k+1)} = Mx^{(k)} + d$. The convergence theory then shows that such an iteration converges from every starting vector exactly when the spectral radius of M is less than one. The last section applies the theory to diagonally dominant, symmetric positive definite, and tridiagonal matrices, including the optimal relaxation parameter for SOR.

Contents

1	Introduction	1
1.1	Why iterate?	1
1.2	Fixed-point form	2
2	Three classical methods	2
2.1	Solving each equation for its own unknown	2
2.2	The Jacobi method	3
2.3	The Gauss–Seidel method	3
2.4	Successive over-relaxation	4
3	Matrix formulation	5
3.1	The splitting $A = D - L - U$	5
3.2	Jacobi	6
3.3	Gauss–Seidel	6
3.4	SOR	6
4	Convergence of iterative methods	7
4.1	Subordinate matrix norms	7
4.2	A sufficient condition	8
4.3	The spectral radius	9
4.4	The convergence theorem	10
5	Iterative methods for special matrices	10
5.1	Strictly diagonally dominant matrices	10
5.2	Matrices with nonpositive off-diagonal entries	11
5.3	A necessary condition on the relaxation parameter	11

5.4	Symmetric positive definite matrices	11
5.5	The optimal relaxation parameter	13

1 Introduction

1.1 Why iterate?

Gaussian elimination solves an $n \times n$ system with about $\frac{2}{3}n^3$ floating-point operations. For a system with $n = 10^6$ unknowns that is roughly 6.7×10^{17} operations. A computer performing 10^{12} operations per second (one teraflop) would need about 6.7×10^5 seconds, or close to eight days, for a single solve. Large systems of this kind usually come from discretized differential equations. Their matrices are *sparse*, with only a handful of nonzero entries per row, but elimination fills in the triangular factors and so loses that advantage.

What counts as a large system has changed with the hardware, as the following rough figures show. Problem sizes have always grown to the limit of what the machines of the day could solve, so the $O(n^3)$ cost of elimination remains a real constraint.

Table 1: The size of a large linear system over time

Year	Typical large n
1950	20
1965	200
1980	2 000
1995	20 000

An *iterative method* gives up on computing $A^{-1}b$ exactly. Starting from a guess $x^{(0)}$, it builds a sequence $x^{(1)}, x^{(2)}, \dots$ with

$$\|x^{(k)} - A^{-1}b\| \rightarrow 0 \quad (k \rightarrow \infty),$$

where each step costs about as much as one multiplication by A . For a sparse matrix that is $O(n)$ operations per step.

1.2 Fixed-point form

Every method in these notes has the form

$$x^{(k+1)} = Mx^{(k)} + d, \quad k = 0, 1, 2, \dots, \quad (1.1)$$

with an *iteration matrix* M and a vector d derived from $Ax = b$. To find them, choose any invertible matrix Q and add and subtract Qx in $Ax - b = 0$:

$$Qx - Qx + Ax - b = 0 \iff Qx = (Q - A)x + b \iff x = (I - Q^{-1}A)x + Q^{-1}b.$$

This suggests the iteration (1.1) with

$$M = I - Q^{-1}A, \quad d = Q^{-1}b. \quad (1.2)$$

Equivalently, in terms of the residual $r^{(k)} = b - Ax^{(k)}$,

$$x^{(k+1)} = x^{(k)} + Q^{-1}r^{(k)}. \quad (1.3)$$

One step therefore needs one product with A and one solve with Q . The matrix M is never formed in practice; it is the object the convergence theory studies.

Proposition 1.1. *If the sequence (1.1) with M and d from (1.2) converges to a vector x^* , then $Ax^* = b$.*

Proof. Let $k \rightarrow \infty$ in (1.1) to obtain $x^* = (I - Q^{-1}A)x^* + Q^{-1}b$, that is, $Q^{-1}(Ax^* - b) = 0$. Since Q is invertible, $Ax^* = b$. $\square$

The choice of Q is the choice of method. Two demands pull in opposite directions:

- Q should be cheap to solve with, for instance diagonal or triangular;
- Q should be close to A , so that $M = I - Q^{-1}A$ is small and the error shrinks quickly.

The extreme $Q = A$ converges in one step but costs a full solve; $Q = I$ costs nothing but rarely converges. The Jacobi, Gauss–Seidel, and SOR methods are three compromises between these extremes.

2 Three classical methods

2.1 Solving each equation for its own unknown

Write the system as

$$\sum_{j=1}^n a_{ij}x_j = b_i, \quad i = 1, \dots, n,$$

and assume that every diagonal entry a_{ii} is nonzero (reorder the equations if necessary). Solving equation i for the unknown x_i gives

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij}x_j - \sum_{j>i} a_{ij}x_j \right), \quad i = 1, \dots, n. \quad (2.1)$$

The exact solution satisfies (2.1). An iteration is obtained by inserting the current guess on the right-hand side and reading a new guess off the left. The methods below differ only in which guess, old or freshly updated, feeds the two sums.

2.2 The Jacobi method

Given $x^{(k)}$, the Jacobi method computes every component of $x^{(k+1)}$ from the old vector:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij}x_j^{(k)} - \sum_{j>i} a_{ij}x_j^{(k)} \right), \quad i = 1, \dots, n. \quad (2.2)$$

The n updates are independent of one another, so a Jacobi sweep parallelizes perfectly. It needs two vectors in memory, the old iterate being read and the new one being written. A practical stopping rule ends the iteration when $\|x^{(k+1)} - x^{(k)}\|$ is small relative to $\|x^{(k+1)}\|$, or after a maximum number of sweeps.

```
import numpy as np
```

```
def jacobi(A, b, x0, tol=1e-8, maxit=200):
    """Jacobi iteration for A x = b, starting from the guess x0."""
    n = len(b)
    x_old = x0.astype(float)
```

```

for it in range(1, maxit + 1):
    x = np.empty(n)
    for i in range(n):
        s = A[i, :i] @ x_old[:i] + A[i, i+1:] @ x_old[i+1:]
        x[i] = (b[i] - s) / A[i, i]
    if np.linalg.norm(x - x_old) <= tol * max(1, np.linalg.norm(x)):
        return x, it
    x_old = x
raise RuntimeError("Jacobi did not converge in maxit iterations")

```

2.3 The Gauss-Seidel method

When $x_i^{(k+1)}$ is computed, the components $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ are already available. The Gauss-Seidel method uses them at once:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n. \quad (2.3)$$

A single vector suffices, since x_i can be overwritten as soon as it is computed. The cost of a sweep is the same as for Jacobi, but the sweep is sequential, and the order in which the unknowns are visited now affects the iterates.

```

def gauss_seidel(A, b, x0, tol=1e-8, maxit=200):
    """Gauss-Seidel iteration: new values are used as soon as they exist."""
    n = len(b)
    x = x0.astype(float).copy()
    for it in range(1, maxit + 1):
        x_old = x.copy()
        for i in range(n):
            # x[:i] already holds sweep k+1, x[i+1:] still holds sweep k
            s = A[i, :i] @ x[:i] + A[i, i+1:] @ x[i+1:]
            x[i] = (b[i] - s) / A[i, i]
        if np.linalg.norm(x - x_old) <= tol * max(1, np.linalg.norm(x)):
            return x, it
    raise RuntimeError("Gauss-Seidel did not converge in maxit iterations")

```

Example 2.1. Consider

$$\begin{bmatrix} 10 & 1 & 1 \\ -1 & 10 & 1 \\ -1 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 10 \\ 8 \end{bmatrix},$$

whose exact solution is $x^* = (1, 1, 1)^T$, and start from $x^{(0)} = (0, 0, 0)^T$. The Jacobi iteration reads

$$\begin{aligned} x_1^{(k+1)} &= (12 - x_2^{(k)} - x_3^{(k)})/10, \\ x_2^{(k+1)} &= (10 + x_1^{(k)} - x_3^{(k)})/10, \\ x_3^{(k+1)} &= (8 + x_1^{(k)} + x_2^{(k)})/10, \end{aligned}$$

and the Gauss–Seidel iteration is identical except that $x_1^{(k+1)}$ replaces $x_1^{(k)}$ in the second and third lines, and $x_2^{(k+1)}$ replaces $x_2^{(k)}$ in the third. In exact arithmetic the first iterates are

$$\begin{aligned} \text{Jacobi: } x^{(1)} &= \left(\frac{6}{5}, 1, \frac{4}{5}\right), & x^{(2)} &= \left(\frac{51}{50}, \frac{26}{25}, \frac{51}{50}\right), & x^{(3)} &= \left(\frac{497}{500}, 1, \frac{503}{500}\right), \\ \text{Gauss–Seidel: } x^{(1)} &= \left(\frac{6}{5}, \frac{28}{25}, \frac{129}{125}\right), & x^{(2)} &= \left(\frac{1231}{1250}, \frac{12441}{12500}, \frac{124751}{125000}\right). \end{aligned}$$

After ten sweeps the Jacobi iterate is $x^{(10)} = \left(\frac{5000000081}{5000000000}, \frac{2500000081}{2500000000}, \frac{5000000081}{5000000000}\right)$, with error about 3×10^{-8} . The Gauss–Seidel iterate $x^{(10)}$ agrees with x^* to about twelve digits. Table 2 lists the error $\|x^{(k)} - x^*\|_\infty$ for these two methods and for the SOR method of the next subsection.

Table 2: Error $\|x^{(k)} - x^*\|_\infty$ for the system of Example 2.1

k	Jacobi	Gauss–Seidel	SOR, $\omega = 0.8$	SOR, $\omega = 1.1$
1	2.0×10^{-1}	2.0×10^{-1}	2.1×10^{-1}	3.2×10^{-1}
2	4.0×10^{-2}	1.5×10^{-2}	4.2×10^{-2}	7.7×10^{-2}
3	6.0×10^{-3}	6.7×10^{-4}	7.6×10^{-3}	1.7×10^{-2}
5	1.8×10^{-4}	1.8×10^{-6}	3.9×10^{-4}	7.0×10^{-4}
10	3.2×10^{-8}	6.0×10^{-13}	1.7×10^{-7}	2.4×10^{-7}

2.4 Successive over-relaxation

SOR first computes the Gauss–Seidel value and then moves from $x_i^{(k)}$ towards it by a fraction ω , the *relaxation parameter*:

$$x_i^{(k+1)} = \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right) + (1 - \omega) x_i^{(k)}, \quad i = 1, \dots, n. \quad (2.4)$$

For $0 < \omega < 1$ the method is called *under-relaxation*, and it damps each step. For $\omega = 1$ it is exactly Gauss–Seidel. For $\omega > 1$ it is *over-relaxation*, an extrapolated step which, for a well-chosen ω , can converge much faster than Gauss–Seidel. Section 5 shows that only $0 < \omega < 2$ can work, and gives the best ω for an important class of matrices.

```
def sor(A, b, x0, omega, tol=1e-8, maxit=200):
    """SOR iteration with relaxation parameter omega (1 = Gauss-Seidel)."""
    n = len(b)
    x = x0.astype(float).copy()
    for it in range(1, maxit + 1):
        x_old = x.copy()
        for i in range(n):
            s = A[i, :i] @ x[:i] + A[i, i+1:] @ x[i+1:]
            x_gs = (b[i] - s) / A[i, i] # Gauss-Seidel value
            x[i] = omega * x_gs + (1 - omega) * x_old[i]
        if np.linalg.norm(x - x_old) <= tol * max(1, np.linalg.norm(x)):
            return x, it
    raise RuntimeError("SOR did not converge in maxit iterations")
```

Example 2.2. For the system of Example 2.1 with $\omega = 0.8$ the SOR iteration is

$$\begin{aligned}x_1^{(k+1)} &= \omega(12 - x_2^{(k)} - x_3^{(k)})/10 + (1 - \omega)x_1^{(k)}, \\x_2^{(k+1)} &= \omega(10 + x_1^{(k+1)} - x_3^{(k)})/10 + (1 - \omega)x_2^{(k)}, \\x_3^{(k+1)} &= \omega(8 + x_1^{(k+1)} + x_2^{(k+1)})/10 + (1 - \omega)x_3^{(k)}.\end{aligned}$$

From $x^{(0)} = 0$ it produces

$$\begin{aligned}x^{(1)} &= \left(\frac{24}{25}, \frac{548}{625}, \frac{12296}{15625}\right) \approx (0.9600, 0.8768, 0.7869), \\x^{(2)} &\approx (1.0189, 0.9939, 0.9584), \\x^{(10)} &\approx (0.99999984, 1.00000017, 1.00000000).\end{aligned}$$

Table 2 shows that neither $\omega = 0.8$ nor $\omega = 1.1$ improves on Gauss–Seidel for this system. Relaxation pays off only with a good choice of ω , and that choice depends on the matrix.

3 Matrix formulation

3.1 The splitting $A = D - L - U$

Write

$$A = D - L - U, \tag{3.1}$$

where $D = \text{diag}(a_{11}, \dots, a_{nn})$ is the diagonal of A , $-L$ is the strictly lower-triangular part of A , and $-U$ is its strictly upper-triangular part:

$$D = \begin{bmatrix} a_{11} & & \\ & \ddots & \\ & & a_{nn} \end{bmatrix}, \quad L = - \begin{bmatrix} 0 & & & \\ a_{21} & 0 & & \\ \vdots & \ddots & \ddots & \\ a_{n1} & \cdots & a_{n,n-1} & 0 \end{bmatrix}, \quad U = - \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ & 0 & \ddots & \vdots \\ & & \ddots & a_{n-1,n} \\ & & & 0 \end{bmatrix}.$$

The signs are chosen so that (3.1) holds exactly. Each method corresponds to a choice of Q built from these three pieces.

3.2 Jacobi

Take $Q = D$. Then $Ax = b$ becomes $Dx = (L + U)x + b$, and the iteration $Dx^{(k+1)} = (L + U)x^{(k)} + b$ is, row by row, exactly (2.2). Hence

$$x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b, \quad M_J = D^{-1}(L + U). \tag{3.2}$$

Indeed $I - D^{-1}A = I - D^{-1}(D - L - U) = D^{-1}(L + U)$, in agreement with (1.2).

3.3 Gauss–Seidel

Take $Q = D - L$. Collecting the new values in (2.3) on the left gives $(D - L)x^{(k+1)} = Ux^{(k)} + b$, so

$$x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b, \quad M_{\text{GS}} = (D - L)^{-1}U. \tag{3.3}$$

Because $D - L$ is lower triangular, solving with it is forward substitution, which is precisely the “use new values at once” sweep.

3.4 SOR

Multiply $Ax = b$ by $\omega \neq 0$ and add $(D - \omega L)x$ to both sides of $0 = \omega b - \omega Ax$:

$$(D - \omega L)x = \omega b - \omega Ax + (D - \omega L)x.$$

Substituting $A = D - L - U$ and simplifying gives $(D - \omega L)x = [(1 - \omega)D + \omega U]x + \omega b$. The SOR iteration (2.4) is the corresponding fixed-point iteration,

$$x^{(k+1)} = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]x^{(k)} + \omega(D - \omega L)^{-1}b, \quad M_{\text{SOR}}(\omega) = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]. \quad (3.4)$$

In the notation of (1.2) this is $Q = \frac{1}{\omega}D - L$. For $\omega = 1$ it reduces to Gauss–Seidel.

Example 3.1. For the tridiagonal matrix

$$A = \begin{bmatrix} 4 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 4 \end{bmatrix}$$

we have $D = 4I$, and L and U carry the entries -1 . The three iteration matrices are

$$M_J = \begin{bmatrix} 0 & -\frac{1}{4} & 0 & 0 \\ -\frac{1}{4} & 0 & -\frac{1}{4} & 0 \\ 0 & -\frac{1}{4} & 0 & -\frac{1}{4} \\ 0 & 0 & -\frac{1}{4} & 0 \end{bmatrix}, \quad M_{\text{GS}} = \begin{bmatrix} 0 & -\frac{1}{4} & 0 & 0 \\ 0 & \frac{1}{16} & -\frac{1}{4} & 0 \\ 0 & -\frac{1}{64} & \frac{1}{16} & -\frac{1}{4} \\ 0 & \frac{1}{256} & -\frac{1}{64} & \frac{1}{16} \end{bmatrix},$$

and, for general ω ,

$$M_{\text{SOR}}(\omega) = \begin{bmatrix} 4 & 0 & 0 & 0 \\ \omega & 4 & 0 & 0 \\ 0 & \omega & 4 & 0 \\ 0 & 0 & \omega & 4 \end{bmatrix}^{-1} \begin{bmatrix} 4(1 - \omega) & -\omega & 0 & 0 \\ 0 & 4(1 - \omega) & -\omega & 0 \\ 0 & 0 & 4(1 - \omega) & -\omega \\ 0 & 0 & 0 & 4(1 - \omega) \end{bmatrix}.$$

For $\omega = \frac{1}{2}$ this evaluates to

$$M_{\text{SOR}}\left(\frac{1}{2}\right) = \begin{bmatrix} \frac{1}{2} & -\frac{1}{8} & 0 & 0 \\ -\frac{1}{16} & \frac{33}{64} & -\frac{1}{8} & 0 \\ \frac{1}{128} & -\frac{33}{512} & \frac{33}{64} & -\frac{1}{8} \\ -\frac{1}{1024} & \frac{33}{4096} & -\frac{33}{512} & \frac{33}{64} \end{bmatrix}.$$

Column j of M_{GS} is the solution of $(D - L)m = Ue_j$, obtained by forward substitution; its first column vanishes because $Ue_1 = 0$.

The iteration matrices can be formed in Python for small experiments. In a real computation one iterates with the functions of the previous section and never builds M .

```
def splitting(A):
    """Return D, L, U with A = D - L - U (L, U strictly triangular)."""
    D = np.diag(np.diag(A))
    L = -np.tril(A, -1)
```

```

U = -np.triu(A, 1)
return D, L, U

def iteration_matrices(A, omega=1.0):
    D, L, U = splitting(A)
    M_J = np.linalg.solve(D, L + U)
    M_GS = np.linalg.solve(D - L, U)
    M_SOR = np.linalg.solve(D - omega * L, (1 - omega) * D + omega * U)
    return M_J, M_GS, M_SOR

rho = lambda M: np.max(np.abs(np.linalg.eigvals(M))) # spectral radius

```

4 Convergence of iterative methods

4.1 Subordinate matrix norms

Throughout this section $\|\cdot\|$ denotes a vector norm on $\mathbb{C}^n$ and the matrix norm it induces,

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}. \quad (4.1)$$

Such a norm is called *subordinate* (or induced). Besides the norm axioms it satisfies

$$\|Ax\| \leq \|A\| \|x\|, \quad \|AB\| \leq \|A\| \|B\|, \quad \|I\| = 1.$$

The norm induced by $\|\cdot\|_\infty$ is the maximum absolute row sum, the one induced by $\|\cdot\|_1$ is the maximum absolute column sum, and the one induced by $\|\cdot\|_2$ is the largest singular value, $\|A\|_2 = \max_i \sigma_i$. The Frobenius norm

$$\|A\|_F^2 = \sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2 = \text{trace}(A^H A) = \sum_{j=1}^n \sigma_j^2$$

is a matrix norm but not a subordinate one, since $\|I\|_F = \sqrt{n}$.

4.2 A sufficient condition

Let x^* satisfy $x^* = Mx^* + d$. Subtracting this from (1.1) eliminates d , and the error $e^{(k)} = x^{(k)} - x^*$ satisfies

$$e^{(k)} = M e^{(k-1)} = \dots = M^k e^{(0)}. \quad (4.2)$$

Taking norms, $\|e^{(k)}\| \leq \|M\|^k \|e^{(0)}\|$.

Theorem 4.1. *Let $M = I - Q^{-1}A$ with Q invertible, and suppose that $\|M\| < 1$ for some subordinate norm. Then:*

1. *the equation $x = Mx + d$ has a unique solution x^* , which is the solution of $Ax = b$, and the iteration (1.1) converges to x^* from every starting vector $x^{(0)}$;*
2. *for every $k \geq 1$,*

$$\|x^{(k)} - x^*\| \leq \frac{\|M\|^k}{1 - \|M\|} \|x^{(1)} - x^{(0)}\|. \quad (4.3)$$

Proof. If $(I - M)z = 0$ then $\|z\| = \|Mz\| \leq \|M\|\|z\|$, which forces $z = 0$. So $I - M = Q^{-1}A$ is invertible, A is invertible, and $x = Mx + d$ has the unique solution $x^* = (Q^{-1}A)^{-1}Q^{-1}b = A^{-1}b$. By (4.2), $\|x^{(k)} - x^*\| \leq \|M\|^k \|x^{(0)} - x^*\| \rightarrow 0$.

For the bound, note that $x^{(l+1)} - x^{(l)} = M(x^{(l)} - x^{(l-1)})$, so $\|x^{(l+1)} - x^{(l)}\| \leq \|M\|^l \|x^{(1)} - x^{(0)}\|$. For $m > k$ the triangle inequality and the geometric series give

$$\begin{aligned} \|x^{(m)} - x^{(k)}\| &\leq \sum_{l=k}^{m-1} \|x^{(l+1)} - x^{(l)}\| \leq \|M\|^k (1 + \|M\| + \dots + \|M\|^{m-1-k}) \|x^{(1)} - x^{(0)}\| \\ &= \|M\|^k \frac{1 - \|M\|^{m-k}}{1 - \|M\|} \|x^{(1)} - x^{(0)}\|. \end{aligned}$$

Letting $m \rightarrow \infty$ yields (4.3). $\square$

The bound (4.3) is *a priori*: after one sweep it predicts how many sweeps suffice for a given tolerance.

Example 4.2. For the matrix of Example 3.1, the absolute row sums of M_J are $\frac{1}{4}, \frac{1}{2}, \frac{1}{2}, \frac{1}{4}$, and those of M_{GS} are $\frac{1}{4}, \frac{5}{16}, \frac{21}{64}, \frac{21}{256}$. Hence

$$\|M_J\|_\infty = \frac{1}{2} < 1, \quad \|M_{GS}\|_\infty = \frac{21}{64} \approx 0.33 < 1,$$

and both methods converge from every starting vector. With $b = A(1, 1, 1, 1)^T = (5, 6, 6, 5)^T$ and $x^{(0)} = 0$, the first Jacobi sweep gives $x^{(1)} = D^{-1}b$ and $\|x^{(1)} - x^{(0)}\|_\infty = 1.5$. The bound (4.3) guarantees an error below 10^{-10} as soon as $3 \cdot 2^{-k} \leq 10^{-10}$, that is, for $k \geq 35$.

The condition $\|M\| < 1$ depends on the norm and is not necessary for convergence. A matrix can have a large norm in one subordinate norm and a small one in another. The spectral radius, introduced next, closes this gap.

4.3 The spectral radius

Definition 4.3. Let A be a complex square matrix.

1. The *adjoint* of A is the matrix A^H with entries $(A^H)_{ij} = \overline{a_{ji}}$, the conjugate transpose. For a real matrix $A^H = A^T$.
2. A is *unitary* if $A^H A = A A^H = I$.
3. The *spectral radius* of A is $\rho(A) = \max_{1 \leq i \leq n} |\lambda_i|$, where $\lambda_1, \dots, \lambda_n$ are the eigenvalues of A .

Theorem 4.4 (Schur). *Every square matrix can be written as $A = UTU^H$ with U unitary and T upper triangular.*

Proof. By induction on n ; the case $n = 1$ is trivial. Let $Ax_1 = \lambda_1 x_1$ with $x_1^H x_1 = 1$. By the Gram-Schmidt process there is a matrix $W = [w_2, \dots, w_n]$ such that $[x_1, W]$ is unitary, that is, $W^H W = I$ and $W^H x_1 = 0$. Then

$$\begin{bmatrix} x_1^H \\ W^H \end{bmatrix} A \begin{bmatrix} x_1 & W \end{bmatrix} = \begin{bmatrix} \lambda_1 x_1^H x_1 & x_1^H A W \\ \lambda_1 W^H x_1 & W^H A W \end{bmatrix} = \begin{bmatrix} \lambda_1 & c^H \\ 0 & C \end{bmatrix},$$

with $c^H = x_1^H A W$ and $C = W^H A W$ of order $n - 1$. By the induction hypothesis $C = \widetilde{W} \widetilde{T} \widetilde{W}^H$ with $\widetilde{W}$ unitary and $\widetilde{T}$ upper triangular. With the unitary matrix $W_1 = \text{diag}(1, \widetilde{W})$,

$$W_1^H \begin{bmatrix} \lambda_1 & c^H \\ 0 & C \end{bmatrix} W_1 = \begin{bmatrix} \lambda_1 & c^H \widetilde{W} \\ 0 & \widetilde{T} \end{bmatrix} =: T$$

is upper triangular. Hence $A = U T U^H$ with the unitary matrix $U = [x_1, W] W_1$. $\square$

Remark 4.5. If U is unitary then $U^H = U^{-1}$, and its columns form an orthonormal basis. In the Schur form, A and T are similar, so the diagonal entries of T are the eigenvalues of A .

Theorem 4.6. *For every square matrix A ,*

$$\rho(A) = \inf_{\|\cdot\|} \|A\|,$$

where the infimum is taken over all subordinate matrix norms.

Proof. First, $\rho(A) \leq \|A\|$ for every subordinate norm: if $Av = \lambda v$ with $v \neq 0$, then $|\lambda| \|v\| = \|Av\| \leq \|A\| \|v\|$.

Conversely, let $0 < \varepsilon \leq 1$ and write $A = U T U^H$ by Theorem 4.4. Put $D_\varepsilon = \text{diag}(\varepsilon, \varepsilon^2, \dots, \varepsilon^n)$ and $S = U D_\varepsilon$. Then

$$S^{-1} A S = D_\varepsilon^{-1} T D_\varepsilon =: T_\varepsilon, \quad (T_\varepsilon)_{ij} = t_{ij} \varepsilon^{j-i},$$

so T_ε is upper triangular with the eigenvalues of A on its diagonal, and $|(T_\varepsilon)_{ij}| \leq \varepsilon |t_{ij}|$ for $j > i$. The vector norm $\|x\|_S = \|S^{-1}x\|_\infty$ induces the matrix norm $\|A\|_S = \|S^{-1} A S\|_\infty = \|T_\varepsilon\|_\infty$, and

$$\|A\|_S \leq \max_i |t_{ii}| + \varepsilon \max_i \sum_{j>i} |t_{ij}| = \rho(A) + c\varepsilon,$$

with c independent of ε . Since ε is arbitrary, $\inf \|A\| \leq \rho(A)$. $\square$

The infimum need not be attained. For example, $\rho\left(\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}\right) = 0$, but the norm of a nonzero matrix is never 0.

4.4 The convergence theorem

Theorem 4.7. *Let M be an $n \times n$ matrix and $d \in \mathbb{R}^n$. The following statements are equivalent.*

1. *The iteration $x^{(k+1)} = Mx^{(k)} + d$ converges for every starting vector $x^{(0)}$.*
2. *$\rho(M) < 1$.*
3. *$\lim_{k \rightarrow \infty} \|M^k\| = 0$.*

Proof. (1) *implies* (2). Suppose the iteration always converges. Its limit x^* satisfies $x^* = Mx^* + d$, so by (4.2), $M^k(x^{(0)} - x^*) \rightarrow 0$ for every $x^{(0)}$, that is, $M^k v \rightarrow 0$ for every real vector v . By linearity the same holds for complex v , applied to its real and imaginary parts. Let v be an eigenvector for an eigenvalue λ . Then $M^k v = \lambda^k v \rightarrow 0$, which forces $|\lambda| < 1$. Hence $\rho(M) < 1$.

(2) *implies* (1). If $\rho(M) < 1$, Theorem 4.6 provides a subordinate norm with $\|M\| < 1$, and the argument of Theorem 4.1 shows that the iteration converges from every $x^{(0)}$ to the unique fixed point of $x = Mx + d$.

(2) *implies* (3). With a subordinate norm satisfying $\|M\| < 1$ we get $\|M^k\| \leq \|M\|^k \rightarrow 0$. All norms on the space of $n \times n$ matrices are equivalent, so $M^k \rightarrow 0$ in every norm.

(3) *implies* (2). Suppose $\|M^k\| \rightarrow 0$ but M has an eigenvalue λ with $|\lambda| \geq 1$ and eigenvector v . For a subordinate norm, $\|M^k\| \geq \|M^k v\|/\|v\| = |\lambda|^k \geq 1$, a contradiction. $\square$

Remark 4.8. Theorem 4.7 also describes the speed of convergence. For large k the error behaves roughly like $C\rho(M)^k$, so each sweep multiplies the error by about $\rho(M)$. The number of sweeps needed to gain p decimal digits is approximately

$$k \approx \frac{p}{-\log_{10} \rho(M)}.$$

For the matrix of Example 3.1, $\rho(M_J) = \frac{1}{2} \cos \frac{\pi}{5} \approx 0.4045$ and $\rho(M_{GS}) = \rho(M_J)^2 \approx 0.1636$. Ten digits therefore take about 25 Jacobi sweeps but only about 13 Gauss–Seidel sweeps.

5 Iterative methods for special matrices

5.1 Strictly diagonally dominant matrices

Recall that A is *strictly diagonally dominant* if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for $i = 1, \dots, n$.

Theorem 5.1. *If A is strictly diagonally dominant, the Jacobi and Gauss–Seidel iterations converge for every starting vector.*

Proof. *Jacobi.* Row i of $M_J = D^{-1}(L + U)$ has entries $-a_{ij}/a_{ii}$ for $j \neq i$ and 0 on the diagonal. Hence

$$\|M_J\|_\infty = \max_i \frac{1}{|a_{ii}|} \sum_{j \neq i} |a_{ij}| < 1,$$

and Theorem 4.1 applies.

Gauss–Seidel. We show $\rho(M_{GS}) < 1$. Let $M_{GS}x = \lambda x$ with $\|x\|_\infty = 1$. Then $Ux = \lambda(D - L)x$, which in row i reads

$$-\sum_{j>i} a_{ij}x_j = \lambda \left(a_{ii}x_i + \sum_{j<i} a_{ij}x_j \right).$$

Choose i with $|x_i| = 1$. Then $\lambda a_{ii}x_i = -\lambda \sum_{j<i} a_{ij}x_j - \sum_{j>i} a_{ij}x_j$, and since $|x_j| \leq 1$ the triangle inequality gives

$$|\lambda| |a_{ii}| \leq |\lambda| \sum_{j<i} |a_{ij}| + \sum_{j>i} |a_{ij}|.$$

By strict dominance $|a_{ii}| - \sum_{j<i} |a_{ij}| > \sum_{j>i} |a_{ij}| \geq 0$, so

$$|\lambda| \leq \frac{\sum_{j>i} |a_{ij}|}{|a_{ii}| - \sum_{j<i} |a_{ij}|} < 1.$$

This holds for every eigenvalue, so $\rho(M_{GS}) < 1$, and Theorem 4.7 applies. $\square$

5.2 Matrices with nonpositive off-diagonal entries

Theorem 5.2 (Stein–Rosenberg). *Suppose $a_{ii} > 0$ for all i and $a_{ij} \leq 0$ for all $i \neq j$. Then exactly one of the following holds:*

1. $\rho(M_J) = \rho(M_{GS}) = 0$;
2. $0 < \rho(M_{GS}) < \rho(M_J) < 1$;
3. $\rho(M_J) = \rho(M_{GS}) = 1$;

4. $1 < \rho(M_J) < \rho(M_{GS})$.

Under this sign pattern the Jacobi and Gauss–Seidel methods therefore converge or diverge together, and when they converge, Gauss–Seidel is faster. For a proof see Varga [3] or Stoer and Bulirsch [2].

5.3 A necessary condition on the relaxation parameter

Theorem 5.3 (Kahan). *Let A have nonzero diagonal entries. Then for every ω ,*

$$\rho(M_{\text{SOR}}(\omega)) \geq |\omega - 1|.$$

Consequently SOR can converge for every starting vector only if $0 < \omega < 2$.

Proof. In (3.4), $D - \omega L$ is lower triangular with diagonal D , and $(1 - \omega)D + \omega U$ is upper triangular with diagonal $(1 - \omega)D$. Therefore

$$\det M_{\text{SOR}}(\omega) = \frac{\det((1 - \omega)D + \omega U)}{\det(D - \omega L)} = \frac{(1 - \omega)^n \det D}{\det D} = (1 - \omega)^n.$$

The determinant is the product of the eigenvalues, so $\rho(M_{\text{SOR}}(\omega))^n \geq |\lambda_1 \cdots \lambda_n| = |1 - \omega|^n$. If $\omega \leq 0$ or $\omega \geq 2$, then $\rho \geq 1$, and Theorem 4.7 rules out convergence for some starting vector. $\square$

5.4 Symmetric positive definite matrices

The next theorem covers a whole family of splittings at once. Here $\langle u, v \rangle = v^H u$ denotes the complex inner product.

Theorem 5.4. *Let A be Hermitian positive definite, and write $A = D - C - C^H$, where D is Hermitian positive definite and C is any matrix. For $\alpha > \frac{1}{2}$ let $Q = \alpha D - C$ and assume Q is invertible. Then $\rho(I - Q^{-1}A) < 1$, so the iteration with $M = I - Q^{-1}A$ converges from every starting vector.*

Proof. Let $Mx = \lambda x$ with $x \neq 0$, and set $y = x - Mx = (1 - \lambda)x = Q^{-1}Ax$. Then

$$Qy = Ax, \tag{5.1}$$

$$(Q - A)y = Ax - Ay = A(x - y) = AMx = \lambda Ax, \tag{5.2}$$

where $Q = \alpha D - C$ and $Q - A = (\alpha - 1)D + C^H$. Take the inner product of (5.1) with y , and the inner product of (5.2) with y :

$$\begin{aligned} \alpha \langle Dy, y \rangle - \langle Cy, y \rangle &= \langle Ax, y \rangle, \\ (\alpha - 1) \langle y, Dy \rangle + \langle y, C^H y \rangle &= \bar{\lambda} \langle y, Ax \rangle. \end{aligned}$$

Since D is Hermitian, $\langle y, Dy \rangle = \langle Dy, y \rangle$, and $\langle y, C^H y \rangle = \langle Cy, y \rangle$. Adding the two equations cancels the terms in C :

$$(2\alpha - 1) \langle Dy, y \rangle = \langle Ax, y \rangle + \bar{\lambda} \langle y, Ax \rangle.$$

Now substitute $y = (1 - \lambda)x$ and use that $\langle Ax, x \rangle$ is real because A is Hermitian:

$$(2\alpha - 1) |1 - \lambda|^2 \langle Dx, x \rangle = [(\overline{1 - \lambda}) + \bar{\lambda}(1 - \lambda)] \langle Ax, x \rangle = (1 - |\lambda|^2) \langle Ax, x \rangle.$$

Here $2\alpha - 1 > 0$, $\langle Dx, x \rangle > 0$, and $\langle Ax, x \rangle > 0$. If $\lambda \neq 1$ the left-hand side is positive, hence $1 - |\lambda|^2 > 0$, that is, $|\lambda| < 1$. If $\lambda = 1$ then $y = 0$, and (5.1) gives $Ax = 0$, which is impossible for a positive definite A and $x \neq 0$. Hence every eigenvalue of M satisfies $|\lambda| < 1$. $\square$

Corollary 5.5 (Ostrowski–Reich). *If A is symmetric positive definite, then $\rho(M_{\text{SOR}}(\omega)) < 1$ for every $0 < \omega < 2$. In particular, the Gauss–Seidel method ($\omega = 1$) converges for every symmetric positive definite matrix.*

Proof. For symmetric A the splitting (3.1) has $U = L^T$, so $A = D - L - L^T$. The diagonal D is positive definite because $a_{ii} = e_i^T A e_i > 0$. Apply Theorem 5.4 with $C = L$ and $\alpha = 1/\omega$. Then $Q = \frac{1}{\omega}D - L$ is the SOR choice of Q , it is invertible (lower triangular with positive diagonal), and $\alpha > \frac{1}{2}$ is exactly $0 < \omega < 2$. $\square$

Remark 5.6. The Jacobi method need not converge for a symmetric positive definite matrix. For

$$A = \begin{bmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{bmatrix}, \quad a = 0.6,$$

the eigenvalues of A are 2.2, 0.4, and 0.4, so A is positive definite. The eigenvalues of M_J are $-2a$, a , and a , so $\rho(M_J) = 1.2 > 1$ and Jacobi diverges. Gauss–Seidel converges by Corollary 5.5. Theorem 5.4 does give a convergent Jacobi-type method with $C = L + U$ and $Q = \alpha D - (L + U)$, but that Q is no longer diagonal.

5.5 The optimal relaxation parameter

For symmetric positive definite tridiagonal matrices the relation between the three methods is known exactly; see Young’s theory of consistently ordered matrices in Varga [3].

Theorem 5.7. *Let A be symmetric positive definite and tridiagonal. Then $\rho(M_{\text{GS}}) = \rho(M_J)^2 < 1$, and the spectral radius of $M_{\text{SOR}}(\omega)$ is minimized at*

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(M_J)^2}}, \quad \text{with} \quad \rho(M_{\text{SOR}}(\omega_{\text{opt}})) = \omega_{\text{opt}} - 1.$$

For $\omega_{\text{opt}} \leq \omega < 2$ one has $\rho(M_{\text{SOR}}(\omega)) = \omega - 1$.

The last statement shows that Kahan’s bound is attained to the right of ω_{opt} . To the left of ω_{opt} the spectral radius increases steeply, so when ω_{opt} must be estimated, it is safer to overestimate it than to underestimate it.

Example 5.8. For the matrix of Example 3.1, $\rho(M_J) = \frac{1}{2} \cos \frac{\pi}{5} \approx 0.4045$, so $\omega_{\text{opt}} \approx 1.0446$ and $\rho(M_{\text{SOR}}(\omega_{\text{opt}})) \approx 0.0446$. Table 3 compares the three methods.

Table 3: Spectral radii and sweeps for ten digits, matrix of Example 3.1

Method	Spectral radius $\rho(M)$	Sweeps for ten digits
Jacobi	0.4045	about 25
Gauss–Seidel	0.1636	about 13
SOR, $\omega = \omega_{\text{opt}} \approx 1.0446$	0.0446	about 7

When $\rho(M_J)$ is close to 1, which is typical for discretized differential equations on fine grids, ω_{opt} is close to 2 and the gain from SOR is dramatic. If $\rho(M_J) = 1 - \delta$ with small δ , then Gauss–Seidel reduces the error per sweep by about $1 - 2\delta$, while optimal SOR reduces it by about $1 - 2\sqrt{2}\delta$. The number of sweeps drops from order $1/\delta$ to order $1/\sqrt{\delta}$.

References

- [1] D. Kincaid and W. Cheney, *Numerical Analysis: Mathematics of Scientific Computing*, 3rd ed., Brooks/Cole, Pacific Grove, CA, 2002.
- [2] J. Stoer and R. Bulirsch, *Introduction to Numerical Analysis*, 3rd ed., Springer, New York, 2002.
- [3] R. S. Varga, *Matrix Iterative Analysis*, 2nd ed., Springer, Berlin, 2000.